

Лабораторна робота N4

Візуалізація об'єктів у JavaFX

Задача. Навчитись створювати графічні зображення та інтерфейс засобами JavaFX. Навчитись оброблять повідомлення від клавіатури та миші. Навчитись створювати діалогові вікна.

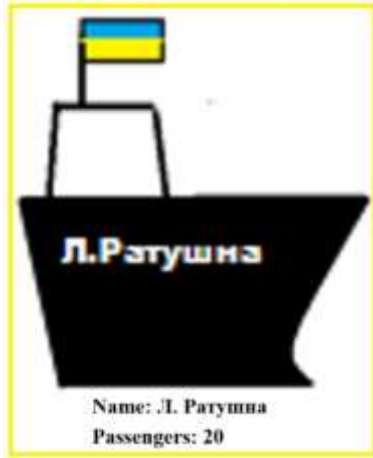
Завдання.

1. Створити проект JavaFX.
2. Додати до проекту клас мікрооб'єкта з попередньої л.р.
3. *Необов'язковий крок.* Нарисувати у MS Paint зображення макрооб'єктів та мікрооб'єктів всіх рівнів, щоб мати уявлення про те як виглядають в графічному представленні всі об'єкти програми. Наприклад так:



Для візуалізації можна використовувати такі графічні примітиви: лінія, прямокутник, еліпс (коло), дуга (арка), багатокутник (полігон), текст, графічне зображення тощо. Графічне представлення кожного мікрооб'єкта має складатися щонайменше з трьох графічних примітивів. При цьому обов'язковим елементом є текстовий примітив (наприклад: прямокутник + еліпс + текст або прямокутник + графічне зображення + текст). Прозорі, білі або невидимі елементи при підрахунку не враховуються!

4. *Необов'язковий крок.* Нарисувати у MS Paint зображення активних мікрооб'єктів всіх рівнів – наприклад так:



5. Зображення активного мікрооб'єкта повинно відрізнятись від зображення неактивного хоча б одним графічним примітивом.
6. При натискуванні лівої кнопки миші на мікрооб'єкті він повинен ставати активним/неактивним. В програмі повинна бути реалізована активація декількох мікрооб'єктів одночасно.
7. Зображення мікрооб'єкта повинно містити графічні примітиви, які показують стан елемента посилального типу, який був доданий до класу мікрооб'єкта, щоб зробити необхідним глибинне копіювання при клонуванні.
8. По певній комбінації клавіш (наприклад Ctrl-C) повинно бути реалізоване копіювання активного мікрооб'єкта.
9. Інтерфейс програми повинен сигналізувати про те, який саме мікрооб'єкт активував користувач. По вибору студента інформація про активний мікрооб'єкт/мікрооб'єкти може виводитись у рядку статусу або робочій області програми. Якщо користувач активував декілька мікрооб'єктів, то інформація про це також має якимось чином відображатися.
10. При натискуванні клавіш-стрілок активні мікрооб'єкти/мікрооб'єкт повинні рухатись у вікні програми.
11. При натискуванні клавіші Insert повинно з'являтись діалогове вікно, яке повинно визначати параметри створюваного мікрооб'єкта. Крім керуючого елемента Button у діалоговому вікні також повинні бути використані як мінімум три з наступного списку: TextField, CheckBox, {ComboBox|ListView|ChoiceBox}, RadioButton. При натисканні кнопки ОК у вказаному діалоговому вікні повинен створюватись мікрооб'єкт з введенними в діалоговому вікні параметрами.
12. При натискуванні правої кнопки миші на мікрооб'єкті повинно з'являтись діалогове вікно, яке дозволяє змінювати параметри вказаного мікрооб'єкта.

13. При натискувань клавіши Delete активні мікрооб'єкти повинні знищуватись. Якщо активного мікрооб'єкта нема – клавіша Delete ігнорується.
14. При натискуванні клавіши Esc повинна відмінатись активація мікрооб'єкта/мікрооб'єктів.
15. Зображення макрооб'єкта складається з графічних примітивів: лінія, прямокутник, еліпс (коло), дуга (арка), багатокутник (полігон), текст, графічне зображення тощо. Графічне представлення кожного макрооб'єкта має складатися щонайменше з трьох графічних примітивів. При цьому обов'язковим елементом є текстовий примітив (наприклад: прямокутник + еліпс + текст або прямокутник + графічне зображення + текст). Прозорі, білі або невидимі елементи при підрахунку не враховуються!.
16. Додати до проекту три або більше макрооб'єктів. Макроб'єкти повинні мати можливість містити певну кількість мікрооб'єктів більше двох. Додати команди рисування макрооб'єктів в програму.
17. Не обов'язково мати можливості додавати/видаляти макрооб'єкти в програму. Цілком достатньо, якщо програма буде мати деяку кількість макрооб'єктів згенеровану при запуску програми і яка буде незмінною протягом роботи програми. Макроб'єкти повинні мати певні позиції в універсальному об'єкті і повинні відображатись на екрані. Позиції макрооб'єктів всередині універсального об'єкта можуть залишатись незмінними протягом роботи програми.
18. В універсальному об'єкті повинно міститись як мінімум один екземпляр кожного оголошеного в темі макрооб'єкта.
19. Мікрооб'єкти можуть або належати одному (або більшій кількості) макрооб'єкту, або не належати жодному. Глядачу має бути зрозуміло в чому полягає приналежність мікрооб'єкта макрооб'єкту, тобто чим поведінка такого мікрооб'єкта відрізняється від поведінки мікрооб'єкта, який не належить жодному макрооб'єкту.
20. У користувача повинна обов'язково бути можливість вилучити будь-який мікрооб'єкт з всіх макрооб'єктів, в результаті чого він не буде належати жодному макрооб'єкту.
21. У користувача повинна обов'язково бути можливість зробити так, щоб виділений мікрооб'єкт став належати одному з наявних макрооб'єктів.

22. В зображенні макрооб'єкта обов'язково повинно виводитись кількість мікрооб'єктів, які йому належать.

Теоретичні відомості

Проект Graph12.

1. Показана обробка клавіш клавіатури:

- стрілки - рух 0-го елемента серед масиву кораблів
- літери та цифри - активують кораблі, назва яких містить дану літеру/цифру
- Delete - видаляє активовані кораблі
- Ctrl+Insert - модальний діалог вводу параметрів нового корабля

2. Показано як створювати об'єкт, який має відображатись у вікні програми

3. Показано, як видаляти з вікна графічні примітиви

4. Як показувати активацію об'єкту

5. Як реалізувати рух об'єктів

6. Як використовувати графічні примітиви: Rectangle, Canvas, як у Canvas рисувати прямокутники, ламані, текст, зафарбовувати полігони

7. Як викликати і ініціалізувати модальні вікна, описані у FXML файлі та відредаговані у Scene Builder.

Як обробляти результат вводу параметрів користувачем.

8. Показані наступні елементи управління в діалогових вікнах:

- TextField
- CheckBox
- ComboBox
- ToggleGroup
- RadioButton
- Button

Main.java

```
package sample;

import javafx.application.Application;
import javafx.application.Platform;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
```

```

import javafx.scene.Group;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.control.Alert;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.stage.Stage;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Random;

class Ship
{
    String name;
    Canvas canvas;
    boolean weight;
    String weightcolor;
    Rectangle rectWeight;
    boolean active;
    Rectangle rectActive;

    public Ship( String name, boolean weight, String weightcolor,
                String shipcolor, double x, double y )
    {
        this.name= name;
        this.weight=weight;

        canvas = new Canvas(230, 225);
        drawShip(canvas, name, weight, shipcolor);

        Main.group.getChildren().add(canvas);

        this.weight = weight;
        this.weightcolor = shipcolor;

        rectWeight = new Rectangle(40,20);
        rectWeight.setFill(Color.valueOf(weightcolor));

        if(this.weight)
        {
            Main.group.getChildren().add(rectWeight);
        }

        rectWeight.setX(x+160);
        rectWeight.setY(y+115);
        canvas.setLayoutX(x);
        canvas.setLayoutY(y);

        active=false;

        rectActive=new Rectangle(234,229);
        rectActive.setFill(Color.TRANSPARENT);
        rectActive.setStrokeWidth(3);
        rectActive.setStroke(Color.BLACK);
    }
}

```

```

        Main.group.getChildren().add(rectActive);
        rectActive.setX(canvas.getLayoutX());
        rectActive.setY(canvas.getLayoutY());
        //System.out.println(canvas);
    }

    public void removeFromScreen()
    {
        if( weight ) Main.group.getChildren().remove(rectWeight);
        Main.group.getChildren().remove(canvas);
        Main.group.getChildren().remove(rectActive);
    }
    public boolean isActive()
    {
        return active;
    }

    public void evaluate(String str)
    {
        System.out.println("Evaluate "+name+" "+str);

        if( name.contains(str) ) {
            if( !active )
            {
                active=true;

                rectActive.setStroke(Color.YELLOW);

                System.out.println("Activated");
            }
        }
        else
        {
            if(active) {
                active=false;

                rectActive.setStroke(Color.BLACK);
                //Main.group.getChildren().remove(rectActive);

                System.out.println("De-activated");
            }
        }
    }

    public void move(double dx, double dy )
    {
        if(this.weight)
        {
            rectWeight.setX(rectWeight.getX()+dx);
            rectWeight.setY(rectWeight.getY()+dy);
        }

        canvas.setLayoutX(canvas.getLayoutX()+dx);
        canvas.setLayoutY(canvas.getLayoutY()+dy);

        rectActive.setX(rectActive.getX()+dx);
        rectActive.setY(rectActive.getY()+dy);
    }

```

```

    }

    public Canvas getCanvas()
    {
        return canvas;
    }

    public static void drawShip( Canvas canvas,
                                String name, boolean weight, String color )
    {
        GraphicsContext gc =
            canvas.getGraphicsContext2D();

        gc.setFill(Color.BLUE);
        gc.fillRect(60, 15, 50, 15);
        gc.setFill(Color.YELLOW);
        gc.fillRect(60, 30, 50, 15);

        gc.setLineWidth(3);
        gc.strokeRect(59,14,52,31);

        gc.beginPath();
        gc.moveTo(59.0, 15.0);
        gc.lineTo(59.0, 75.0);
        gc.stroke();

        gc.beginPath();
        gc.moveTo(40.0, 135.0);
        gc.lineTo(50.0, 75.0);
        gc.lineTo(110.0, 75.0);
        gc.lineTo(130.5, 135.0);
        gc.stroke();

        double[] xrubka={40.0, 50.0, 110.0,130.5, 40.0};
        double[] yrubka={135.0, 75.0, 75.0, 135.0, 135.0};

        gc.setFill(Color.WHITE);
        gc.fillPolygon(xrubka, yrubka, xrubka.length);

        gc.setFill(Color.BLACK);

        double []xpoints={ 30.0, 230.0, 205.0, 208.0, 212.0, 220.0, 45.0, 30.0 };
        double []ypoints={135.0, 135.0, 195.0, 205.0, 215.0, 225.0, 225.0, 135.0 };

        gc.setFill(Color.valueOf(color));
        gc.fillPolygon(xpoints, ypoints, xpoints.length);

        gc.setFont(new Font(30));
        gc.setFill(Color.WHITE);
        gc.fillText(name,60.0, 165.0);
    }
}

public class Main extends Application {

```

```

public static Stage primaryStage;
public static Scene scene;
public static Random rnd= new Random();
public static Group group;

public static ArrayList<Ship> ships = new ArrayList<Ship>();

public static void addNewShip(String name, boolean weight,
                               String weightcolor, String shipcolor )
{
    ships.add(new Ship(name, weight, weightcolor, shipcolor,
        (double)rnd.nextInt(670), (double)rnd.nextInt(475)) );
}

@Override
public void start(Stage primaryStage) throws Exception{

    Main.primaryStage = primaryStage;
    Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
    primaryStage.setTitle("Hello World");

    Scene input = new Scene(root,500,500);

    group = new Group();

    Rectangle rect = new Rectangle(900, 700, Color.BLUE);

    Rectangle r = new Rectangle();
    r.setX(250);
    r.setY(250);
    r.setWidth(200);
    r.setHeight(100);
    r.setArcWidth(20);
    r.setArcHeight(20);
    r.setFill(Color.WHITE);

    group.getChildren().addAll(rect,r);

    ships.add( new Ship( "Л.Патышна", true, "Black","green",
        0,0) );

    scene = new Scene(group,900, 700);

    primaryStage.setScene(scene);

    scene.setOnKeyPressed(new EventHandler<KeyEvent>() {
        @Override
        public void handle(KeyEvent event) {
            double x=0;
            double y=0;

            double delta=5.0;

            if(event.isShiftDown())delta*=10.0;

            if (event.getCode().isLetterKey() || event.getCode().isDigitKey())

```

```

        || event.getCode() == KeyCode.MINUS)
    {
        for ( Ship ship: ships )
        {
            ship.evaluate(event.getText());
        }
    }

    switch(event.getCode()) {

        case INSERT:
            if(event.isControlDown()) {
                try {
                    DialogNewShip.display();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }

            break;

        case DELETE:
            for(int i=ships.size()-1; i>=0; --i )
            {
                Ship ship= ships.get(i);
                if( ship.isActive() )
                {
                    ship.removeFromScreen();
                    ships.remove(i);
                }
            }
            break;

        case UP:
            y-=delta;break;
        case DOWN:
            y+=delta;break;
        case LEFT:
            x-=delta; break;
        case RIGHT:
            x+=delta; break;
    }
    ships.get(0).move(x,y);

    //System.out.println(canvas.getBoundsInParent());
    //System.out.println(r.getBoundsInParent());
    if(
ships.get(0).getCanvas().getBoundsInParent().intersects(r.getBoundsInParent()))
    {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);

        alert.setTitle("УВАГА!!!");
        alert.setHeaderText("Зіткнення з айсбергом!");

        alert.showAndWait();
        //Platform.exit();
    }

```

```

        }
    });

    //primaryStage.setScene(new Scene(root, 300, 275));
    primaryStage.show();
}

public static void main(String[] args) {
    Launch(args);
}
}

```

DialogNewShip.java

```

package sample;

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Modality;
import javafx.stage.Stage;

import java.io.IOException;

public class DialogNewShip {

    public static Stage window=null;
    public static Scene scene;
    public static void display() throws IOException {

        Parent root = FXMLLoader.load(Main.class.getResource("sample.fxml"));
        window = new Stage();
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle("Створіть новий корабель");

        scene = new Scene(root, 440, 358);
        window.setScene(scene);

        window.showAndWait();

    }
}

```

Controller.java

```

package sample;

import javafx.event.ActionEvent;
import javafx.fxml.Initializable;
import javafx.scene.control.*;

import java.net.URL;

```

```

import java.util.ResourceBundle;

public class Controller implements Initializable {
    public TextField strName;
    public CheckBox chckWeight;
    public ComboBox strColor;
    public ToggleGroup tgr;

    @Override
    public void initialize(URL location, ResourceBundle resources) {
        strName.setText("Л.Патушна");
        chckWeight.setSelected(true);

        strColor.getItems().addAll("Yellow", "Red", "Green", "Blue");

        System.out.println("Controler.initialize");
    }

    public void pressOK(ActionEvent actionEvent) {
        String strN= strName.getText();

        if( ! ( (strN != null) && (strN.length()>0) ) )
        {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);

            alert.setTitle("УВАГА!!!");
            alert.setHeaderText("Введіть ім'я!");

            alert.showAndWait();

            return;
        }

        boolean chckW= chckWeight.isSelected();

        if( strColor.getValue()==null )
        {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);

            alert.setTitle("УВАГА!!!");
            alert.setHeaderText("Оберіть колір!");

            alert.showAndWait();

            return;
        }
        String strC = strColor.getValue().toString();

        RadioButton rezrb = (RadioButton)tgr.getSelectedToggle();
        System.out.println("Колір вантажу "+rezrb.getText());

        Main.addNewShip(strN, chckW, rezrb.getText() , strC );

        DialogNewShip.window.close();
    }

    public void pressCancel(ActionEvent actionEvent) {
        Main.primaryStage.setScene(Main.scene);
    }
}

```

```
}
```

sample.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<Pane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity"
  prefHeight="358.0" prefWidth="440.0" xmlns="http://javafx.com/javafx/10.0.2-internal"
  xmlns:fx="http://javafx.com/fxml/1" fx:controller="sample.Controller">
  <children>
    <Label layoutX="94.0" layoutY="52.0" text="Назва" />
    <TextField fx:id="strName" layoutX="156.0" layoutY="48.0" />
    <CheckBox fx:id="chckWeight" layoutX="156.0" layoutY="105.0" mnemonicParsing="false" />
    <Label layoutX="54.0" layoutY="105.0" prefHeight="17.0" prefWidth="80.0" text="Чи є вантаж?" />
    <ComboBox fx:id="strColor" layoutX="155.0" layoutY="159.0" prefWidth="150.0" />
    <Label layoutX="52.0" layoutY="163.0" text="Колір корабля" />
    <Button fx:id="btnOK" layoutX="133.0" layoutY="296.0" mnemonicParsing="false"
      onAction="#pressOK" text="OK" />
    <Button fx:id="btnCancel" layoutX="256.0" layoutY="296.0" mnemonicParsing="false"
      onAction="#pressCancel" text="Cancel" />
    <RadioButton layoutX="81.0" layoutY="253.0" mnemonicParsing="false" selected="true"
      text="Red">
      <toggleGroup>
        <ToggleGroup fx:id="tgr" />
      </toggleGroup>
    </RadioButton>
    <RadioButton layoutX="184.0" layoutY="253.0" mnemonicParsing="false" text="Green"
      toggleGroup="$tgr" />
    <RadioButton layoutX="296.0" layoutY="253.0" mnemonicParsing="false" text="Blue"
      toggleGroup="$tgr" />
    <Label layoutX="179.0" layoutY="217.0" text="Колір вантажу:" />
  </children>
</Pane>
```



Проект ілюструє додання кнопок у вікно та обробку повідомлень від них, обробку повідомлень від клавіатури, виклик інформаційного підвікна Alert та завершення додатку по певній умові

```
//Main.java
//Проект untitled9
import javafx.application.Application;
import javafx.application.Platform;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.scene.Group;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class Main extends Application
{

    Button button1;
    Button button2;
    Button button3;
    Button button4;
    Rectangle rectmine;

    Rectangle rectagent;

    @Override
    public void start(Stage primaryStage) throws Exception{
        //Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));
        //primaryStage.setTitle("Hello World");
        //primaryStage.setScene(
        //    new Scene(root, 300, 275));

        rectmine =new Rectangle();
        rectmine.setX(300);
        rectmine.setY(10);
        rectmine.setHeight(50);
        rectmine.setWidth(50);
        rectmine.setArcHeight(10);
        rectmine.setArcWidth(10);
        rectmine.setFill(Color.RED);

        rectagent= new Rectangle();
        rectagent.setX(150);
        rectagent.setY(300);
        rectagent.setHeight(50);
```

```

rectagent.setWidth(50);
rectagent.setArcHeight(10);
rectagent.setArcWidth(10);
rectagent.setFill(Color.GREEN);

button1 = new Button();
button1.setText("Up");
button1.setFont(new Font("Times New Roman", 30));
button1.setOnAction(
    new EventHandler<ActionEvent>() {

        @Override
        public void handle(ActionEvent event) {

            double y = rectmine.getY();
            y-=10;
            rectmine.setY(y);
        }
    });

button2 = new Button();
button2.setText("Right");
button2.setFont(new Font(30));
button2.setOnAction(
    new EventHandler<ActionEvent>() {

        @Override
        public void handle(ActionEvent event) {
            double x = rectmine.getX();
            x+=10;
            rectmine.setX(x);
        }
    });

button3 = new Button();
button3.setText("Down");
button3.setFont(new Font(30));
button3.setOnAction(
    new EventHandler<ActionEvent>() {

        @Override
        public void handle(ActionEvent event) {
            double y = rectmine.getY();
            y+=10;
            rectmine.setY(y);
        }
    });

button4 = new Button();
button4.setText("Left");
button4.setFont(new Font(30));
button4.setOnAction(
    new EventHandler<ActionEvent>() {

        @Override
        public void handle(ActionEvent event) {
            double x = rectmine.getX();

```

```

        x-=10;
        rectmine.setX(x);
    }
});

//StackPane layout= new StackPane();
Group group = new Group();
group.getChildren().addAll(button1,
    button2,button3,button4);

group.getChildren().add(rectmine);
group.getChildren().add(rectagent);

button2.relocate(0,70);
button3.relocate(0,140);
button4.relocate(0,210);

Scene scene = new Scene(group, 700,500);

scene.setOnKeyPressed(new KeyPressedHandler());

primaryStage.setScene(scene);
primaryStage.show();
}

private class KeyPressedHandler implements EventHandler<KeyEvent>
{
    @Override
    public void handle(KeyEvent event) {
        double x, y;

        x= rectagent.getX();
        y= rectagent.getY();

        if (event.getCode().equals(KeyCode.UP) )
        {
            y-=10;
        }

        if (event.getCode().equals(KeyCode.RIGHT) )
        {
            x+=10;
        }

        if (event.getCode().equals(KeyCode.DOWN) )
        {
            y+=10;
        }

        if (event.getCode().equals(KeyCode.LEFT) )
        {
            x-=10;
        }
    }
}

```

```

        rectagent.setX(x);
        rectagent.setY(y);

        if( rectagent.intersects(
            rectmine.getX(),
            rectmine.getY(),
            rectmine.getWidth(),
            rectmine.getHeight() )
        )
        {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setTitle("Вибух!!!");
            alert.setHeaderText("Ви програли");
            alert.setContentText("Підірвались на міні!");

            alert.showAndWait();
            Platform.exit();
        }

    /*
        switch (event.getCode()) {
            case W:
                rect.
                break;
            case S:
                leftPaddle.moveDown();
                break;
            default:
                break;
        }
    */

        }

    }

    public static void main(String[] args)
    {
        launch(args);
    }
}

```

Проект ілюструє обробку повідомлень від клавіатури та миші, таймер.
Промодельовано мікрооб'єкти та їх взаємодія з клавіатурою та мишою

```

//Main.java
//Проект untitled12
import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.geometry.Point2D;
import javafx.scene.Group;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.input.MouseEvent;

```

```

import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

class MyRect {
    Rectangle r;
    boolean isActive;

    public MyRect(Rectangle r, boolean ia) {
        this.r = r;
        isActive = ia;

        if (ia)
            r.setFill(Color.RED);
        else
            r.setFill(Color.BLUE);
    }
    public Rectangle getRectangle()
    {
        return r;
    }

    public void MouseClick(double x, double y)
    {
        Point2D m = new Point2D(x,y);

        if( r.contains(m) )
        {
            isActive= !isActive;

            if (isActive)
                r.setFill(Color.RED);
            else
                r.setFill(Color.BLUE);
        }

    }

    public void Left()
    {
        if(!isActive) return;

        double x = r.getX()-10;
        r.setX(x);
    }

    public void Up()
    {
        if(!isActive) return;

        double y = r.getY()-10;
        r.setY(y);
    }

    public void Right()
    {
        if(!isActive) return;

        double x = r.getX()+10;
        r.setX(x);
    }
}

```

```

public void Down()
{
    if(!isActive) return;

    double y = r.getY()+10;
    r.setY(y);
}

}

public class Main extends Application {

    static MyRect [] heroes;

    Rectangle rect;
    int dx=10, dy=10;

    Scene scene;
    @Override
    public void start(Stage primaryStage) throws Exception{

        heroes = new MyRect[3];

        heroes[0]= new MyRect(
            new Rectangle(10,20,100,100),
            false );

        heroes[1]= new MyRect(
            new Rectangle(200,10,100,100),
            false );

        heroes[2]= new MyRect(
            new Rectangle(10,200,100,100),
            true );

        Group grp = new Group();

        for( MyRect mr:heroes )
            grp.getChildren().add( mr.getRectangle() );

        rect = new Rectangle(50, 50, 100, 100);
        grp.getChildren().add(rect);

        AnimationTimer timer = new AnimationTimer() {

            @Override
            public void handle(long now) {
                double d = now / 5_000_000_000d;

                boolean flg=true;
                double newX = rect.getX() + dx;

                if( (newX<0) ||
                    (newX+rect.getWidth()
                     >=scene.getWidth()) )
                {
                    dx=-1*dx;

```

```

        flg=false;
    }

    double newY = rect.getY() + dy;

    if( (newY<0) ||
        (newY+rect.getHeight()>=scene.getHeight()) )
    {
        dy=-1*dy;
        flg=false;
    }

    if(flq)
    {
        rect.setX(newX);
        rect.setY(newY);
    }

    }

};

scene= new Scene(grp, 600, 500);

scene.setOnKeyPressed(new KeyPressedHandler());

scene.setOnMousePressed(new EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent event) {

        for( MyRect mr:heroes )
            mr.MouseClick(event.getX(),event.getY());

    }
});

timer.start();

primaryStage.setScene(scene);
primaryStage.show();
}

private class KeyPressedHandler implements EventHandler<KeyEvent> {

    @Override
    public void handle(KeyEvent event) {

        if (event.getCode().equals(KeyCode.UP)) {
            for( MyRect r :heroes)
                r.Up();
        }

        if (event.getCode().equals(KeyCode.RIGHT)) {

```

```

        for( MyRect r :heroes)
            r.Right();

    }

    if (event.getCode().equals(KeyCode.DOWN)) {
        for( MyRect r :heroes)
            r.Down();

    }

    if (event.getCode().equals(KeyCode.LEFT)) {
        for( MyRect r :heroes)
            r.Left();

    }

}

}

public static void main(String[] args) {
    launch(args);
}
}

```

Перший проект. Демонструє можливість переключення між різними сценами. Демонструє використання модального діалогового вікна, яке вводить інформацію і повертає її у викликаючу функцію.

```
//Main.java
```

```
//Проект untitled15
```

```

package sample;

import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.scene.layout.VBox;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class Main extends Application {

    Stage window;

    Scene scenel;
    Scene scene2;

    @Override
    public void start(Stage primaryStage) throws Exception{
        // Parent root = FXMLLoader.load(getClass().getResource("sample.fxml"));

```

```

primaryStage.setTitle("Hello World");
//    primaryStage.setScene(new Scene(root, 300, 275));
window = primaryStage;

//scene1
Label label1 = new Label("This is the first scene!");
label1.setFont(new Font(45));
Button button1 = new Button("Go to scene2");
button1.setFont(new Font(45));
button1.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        window.setScene(scene2);
    }
});

Button button3 = new Button("Alert Window");
button3.setFont(new Font(45));
button3.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        System.out.println(AlertWindow.display("Input", "Are you sure?"));
    }
});

VBox layout1 = new VBox(20);
layout1.getChildren().addAll(label1, button1, button3);
scene1 = new Scene(layout1, 500, 350);

//scene2
Button button2 = new Button("Let's go to scene1");
button2.setFont(new Font(45));
button2.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        window.setScene(scene1);
    }
});

StackPane layout2 = new StackPane();
layout2.getChildren().add(button2);
scene2 = new Scene(layout2, 500, 600);

window.setScene(scene1);
window.show();
}

public static void main(String[] args)
{
    launch(args);
}
}

```

//AlertWindow.java

```

package sample;

import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;

```

```

import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.scene.layout.VBox;
import javafx.scene.text.Font;
import javafx.stage.Modality;
import javafx.stage.Stage;

public class AlertWindow {

    static boolean answer=false;

public static boolean display(String title, String message)
{
    Stage window = new Stage();
    window.initModality(Modality.APPLICATION_MODAL);
    window.setTitle(title);

    Label label1=new Label(message);
    label1.setFont(new Font(45));

    Button yesButton = new Button("Yes");
    yesButton.setFont(new Font(45));
    yesButton.setOnAction(new EventHandler<ActionEvent>() {
        @Override
        public void handle(ActionEvent event) {
            answer=true;
            window.close();
        }
    });

    Button noButton = new Button("No");
    noButton.setFont(new Font(45));
    noButton.setOnAction(new EventHandler<ActionEvent>() {
        @Override
        public void handle(ActionEvent event) {
            answer=false;
            window.close();
        }
    });

    VBox layout = new VBox(5);
    layout.getChildren().addAll(label1,yesButton,noButton);

    Scene scene = new Scene(layout, 500,500);
    window.setScene(scene);

    window.showAndWait();

    return answer;
}
}

```

Ще один проект. Прямокутники можуть активуватись/деактивуватись натискуваннями миші. Клавiши-стрiлки рухають тiльки активованi прямокутники. По клавiшi Insert з'являється дiалогове вiкно, де вводяться координати нового прямокутника.

```
//Main.java
//Проект untitled12

package sample;
import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.geometry.Point2D;
import javafx.scene.Group;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

import java.util.Arrays;

class MyRect {
    Rectangle r;
    boolean isActive;

    public MyRect(Rectangle r, boolean ia) {
        this.r = r;
        isActive = ia;

        if (ia)
            r.setFill(Color.RED);
        else
            r.setFill(Color.BLUE);
    }
    public Rectangle getRectangle()
    {
        return r;
    }

    public void MouseClick(double x, double y)
    {
        Point2D m = new Point2D(x,y);

        if( r.contains(m) )
        {
            isActive= !isActive;

            if (isActive)
                r.setFill(Color.RED);
            else
                r.setFill(Color.BLUE);
        }
    }
}
```

```

    }

    public void Left()
    {
        if(!isActive) return;

        double x = r.getX()-10;
        r.setX(x);
    }

    public void Up()
    {
        if(!isActive) return;

        double y = r.getY()-10;
        r.setY(y);
    }

    public void Right()
    {
        if(!isActive) return;

        double x = r.getX()+10;
        r.setX(x);
    }

    public void Down()
    {
        if(!isActive) return;

        double y = r.getY()+10;
        r.setY(y);
    }
}

public class Main extends Application {

    static MyRect [] heroes;

    Rectangle rect;
    int dx=10, dy=10;
    Group grp;

    Scene scene;
    @Override
    public void start(Stage primaryStage) throws Exception{

        heroes = new MyRect[3];

        heroes[0]= new MyRect (
            new Rectangle(10,20,100,100),
            false );

        heroes[1]= new MyRect (
            new Rectangle(200,10,100,100),
            false );

        heroes[2]= new MyRect (

```

```

        new Rectangle(10,200,100,100),
        true );
grp = new Group();

for( MyRect mr:heroes )
    grp.getChildren().add( mr.getRectangle() );

rect = new Rectangle(50, 50, 100, 100);
grp.getChildren().add(rect);

AnimationTimer timer = new AnimationTimer() {

    @Override
    public void handle(long now) {
        double d = now / 5_000_000_000d;

        boolean flg=true;
        double newX = rect.getX() + dx;

        if( (newX<0) ||
            (newX+rect.getWidth()
             >=scene.getWidth()) )
        {
            dx=-1*dx;
            flg=false;
        }

        double newY = rect.getY() + dy;

        if( (newY<0) ||
            (newY+rect.getHeight()>=scene.getHeight()) )
        {
            dy=-1*dy;
            flg=false;
        }

        if(flg)
        {
            rect.setX(newX);
            rect.setY(newY);
        }

    }

};

scene= new Scene(grp, 800, 800);

scene.setOnKeyPressed(new KeyPressedHandler());

scene.setOnMousePressed(new EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent event) {

        for( MyRect mr:heroes )
            mr.MouseClick(event.getX(),event.getY());

    }

});

```

```

        timer.start();

        primaryStage.setScene(scene);
        primaryStage.show();
    }

    private class KeyPressedHandler implements EventHandler<KeyEvent> {

        @Override
        public void handle(KeyEvent event) {

            if (event.getCode().equals(KeyCode.INSERT)) {
                {
                    if( NewRectDlg.display("New Rectangle","Enter coords"))
                    {
                        Main.heroes= Arrays.copyOf(Main.heroes, Main.heroes.length+1);

                        Rectangle r=new Rectangle
                                (NewRectDlg.x,
                                 NewRectDlg.y,
                                 NewRectDlg.wx,
                                 NewRectDlg.wy);

                        Main.heroes[Main.heroes.length-1]=
                                new MyRect(r,false);

                        grp.getChildren().add(r);
                    }
                }

            }

            if (event.getCode().equals(KeyCode.UP)) {
                for( MyRect r :heroes)
                    r.Up();
            }

            if (event.getCode().equals(KeyCode.RIGHT)) {
                for( MyRect r :heroes)
                    r.Right();
            }

            if (event.getCode().equals(KeyCode.DOWN)) {
                for( MyRect r :heroes)
                    r.Down();
            }

            if (event.getCode().equals(KeyCode.LEFT)) {
                for( MyRect r :heroes)
                    r.Left();
            }
        }
    }

```

```

    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

//NewRectDlg.java

```

package sample;
import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.fxml.FXMLLoader;
import javafx.geometry.Point2D;
import javafx.scene.Group;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.stage.Modality;
import javafx.stage.Stage;

public class NewRectDlg {

    static boolean answer = false;
    public static double x = 0;
    public static double y = 0;
    public static double wx = 0;
    public static double wy = 0;

    public static boolean display(String title, String message)
    {
        Stage window = new Stage();
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle(title);

        Label label0 = new Label(message);
        label0.setFont(new Font(24));

        Label label1 = new Label("X:");
        label1.setFont(new Font(24));
        TextField textField1 = new TextField();
        textField1.setFont(new Font(24));

        Label label2 = new Label("Y:");
        label2.setFont(new Font(24));
        TextField textField2 = new TextField();
        textField2.setFont(new Font(24));
    }
}

```

```

Label label3=new Label("WIDTH:");
label3.setFont(new Font(24));
TextField textField3 = new TextField();
textField3.setFont(new Font(24));

Label label4=new Label("HEIGHT:");
label4.setFont(new Font(24));
TextField textField4 = new TextField();
textField4.setFont(new Font(24));

Button yesButton = new Button("Yes");
yesButton.setFont(new Font(24));
yesButton.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        answer=true;

        x=Integer.parseInt(textField1.getText());
        y=Integer.parseInt(textField2.getText());
        wx=Integer.parseInt(textField3.getText());
        wy=Integer.parseInt(textField4.getText());

        window.close();
    }
});

VBox layout = new VBox(5);
layout.getChildren().addAll
    (label0,
     label1,textField1,
     label2,textField2,
     label3,textField3,
     label4,textField4,
     yesButton
    );

Scene scene = new Scene(layout, 700,500);
window.setScene(scene);

window.showAndWait();

return answer;
}
}

```

Контрольні питання

1. Об'єкт Group. Навіщо потрібен, як створюється, як зв'язується з іншими об'єктами програми.
2. Об'єкт Scene. Навіщо потрібен, як створюється, як зв'язується з іншими об'єктами програми.

3. Об'єкт Stage. Навіщо потрібен, як створюється, як зв'язується з іншими об'єктами програми.
4. Об'єкт Pane. Навіщо потрібен, як створюється, як зв'язується з іншими об'єктами програми.
5. Об'єкт VBox. Навіщо потрібен, як створюється, як зв'язується з іншими об'єктами програми.
6. Як додати кнопку Button у вікно. Як призначити функцію-обробник події натискування на кнопку.
7. Як додати обробник події натискування лівої кнопки миші у сцену. Як отримати координати події.
8. Як додати обробник події натискування клавіші на клавіатурі. Як отримати код клавіші.
9. Як рисувати прямокутники Rectangle у вікні програми. Як задавати колір, позицію та розмір
11. Як рисувати лінію Line. Як задавати координати, колір, стиль
12. Як вивести модальне вікно-повідомлення Alert
13. Як завершити програму
14. Як створити іншу сцену і переключитись на неї
15. Як створити модальне діалогове вікно і забезпечити обмін інформації з викликаючою функцією
16. Для чого використовується елемент управління Label і як він настраюється і додається у діалогове вікно.
17. Для чого використовується елемент управління CheckBox і як він настраюється і додається у діалогове вікно. Як отримати результат вибору
18. Для чого використовується елемент управління TextField і як він додається у діалогове вікно. Як отримати введену інформацію
19. Як перетворити String на int
20. Як перетворити int на String
21. Генерація випадкових цілих чисел в діапазоні
22. Генерація випадкових дійсних чисел в діапазоні
23. Вивод тексту в певну позицію в JavaFX
24. Як перевірити перетинання двох прямокутників
25. Як визначити, яка клавіша була натиснута
26. Для чого використовується елемент управління ListView, і як він настраюється і додається у діалогове вікно. Як отримати результат вибору
27. Для чого використовується елемент управління ChoiceBox , і як він настраюється і додається у діалогове вікно. Як отримати результат вибору
28. Для чого використовується елемент управління ComboBox і як він настраюється і додається у діалогове вікно. Як отримати результат вибору
29. Для чого використовуються елементи управління ToggleGroup та RadioButton і як вони настраюються і додаються у діалогове вікно. Як отримати результат вибору
30. Як формувати діалогове вікно з допомогою FXML файла та Scene Builder. Як настроїти клас Controller. Як додавати елементи класу Controller, які відповідають елементам управління діалогового вікна. Як додавати методи класу Controller, які обробляють повідомлення від елементів управління діалогового вікна
31. Як при виклику ініціалізувати елементи управління діалогового вікна, створеного з допомогою FXML файла та Scene Builder.